

Defensa de QTZ2

Análisis del problema

Se requiere desarrollar un prototipo de consola para gestionar los componentes de la misión del satélite Quetzal-2. El sistema representará una misión en progreso mediante una carga inicial de al menos 10 módulos, almacenados en una única lista polimórfica.

Los módulos se clasifican en tres tipos:

- Módulos de vuelo: cámaras o sensores que recolectan datos científicos.
- Módulos de tierra: antenas que descargan la información del satélite.
- Módulos de energía: paneles solares o baterías que mantienen el sistema encendido.

Todos comparten un ID, nombre, estado de salud y costo de construcción. Cada tipo presenta un comportamiento distinto al ejecutar ``procesarCiclo()``, por lo que se utilizarán herencia y polimorfismo para representar sus características comunes y sus diferencias.

Requisitos funcionales

1. Cargar inicialmente al menos 10 módulos, incluyendo los tres tipos.
2. Listar todos los módulos con sus características.
3. Buscar módulos por ID o por nombre y mostrar su información.
4. Ordenar los módulos por costo de construcción mediante la interfaz ``Comparable``.
5. Ofrecer un menú de consola para acceder a estas operaciones.

Entradas, procesos y salidas

Elemento: Descripción

Entradas: Opción del menú, ID o nombre del módulo que se desea buscar.

Procesos: Carga inicial, consulta de módulos, búsqueda y ordenamiento por costo.

Salidas: Listado de módulos, resultados de búsqueda y mensajes ante opciones inválidas o búsquedas sin coincidencias.

Restricciones y consideraciones

La información presentada debe corresponder a los datos registrados en la carga inicial. El menú requerido no contempla construir módulos nuevos ni avanzar ciclos de simulación.

El programa aplicará el patrón MVC para separar la lógica y los datos de la interacción por consola. También utilizará encapsulación, sobrescritura de `toString()` y sobrecarga de métodos para las búsquedas por ID y por nombre.

Como criterios de validación, se propone que los IDs sean únicos, los costos no sean negativos y las entradas incorrectas se manejen sin interrumpir el programa.

Clases propuestas

- Modulo (abstracta): contiene ID, nombre, salud y costo de construcción. Implementa `Comparable<Modulo>` y declara `procesarCiclo()`.
- ModuloVuelo: hereda de `Modulo` y representa cámaras o sensores que recolectan datos.
- ModuloTierra: hereda de `Modulo` y representa antenas que descargan información.
- ModuloEnergia: hereda de `Modulo` y representa paneles solares que generan energía, elegidos para este prototipo.
- RecursosMision: administra la energía disponible, los datos pendientes y los datos descargados.
- `Mision`: contiene la lista polimórfica y realiza la carga inicial, las búsquedas y el ordenamiento.
- VistaConsola: muestra el menú, recibe entradas y presenta resultados.
- ControladorMision: coordina las operaciones entre la vista y el modelo.
- Principal: crea los objetos necesarios e inicia el programa.

